

Certified braid kernels and linear Markov descent

Minority-sign parameters for exact and
quasipolynomial unknot recognition

Research report prepared for the ProveIt project

8 October 2026

Abstract

We develop a structural preprocessing route for explicit Artin braid words whose closures are knots. For a word of length n on b strands, let $g_B = (n - b + 1)/2$ be the genus of its canonical braid surface and let k be the smaller of its positive and negative letter counts. Splitting at every generator index with a unique occurrence produces a certified connected-sum decomposition with at most $4g_B$ surviving crossings. Applying the Bennequin obstruction separately to the factors either proves nontriviality or gives a closure-preserving kernel with at most $4\kappa \leq 4k$ crossings and at most $2\kappa + 1$ strands, where κ is the sum of the factors' minority counts. This yields a complete fixed-parameter algorithm with time $\text{poly}(n)2^{O(k)}$, and the stronger factorwise bound $\text{poly}(n)2^{O(k_*)}$, where k_* is the largest surviving factor parameter. In particular, $k_* = O(\log^2 n)$ gives $n^{O(\log n)}$ time.

A second contribution replaces repeated global scans in singleton endpoint Markov descent by an amortized-linear algorithm with a separately replayed, linear-size trace. The package includes a standard-library implementation, a small-instance reduced Khovanov oracle, certificate verifiers, finite exhaustive checks, and paired measurements. The singleton topological observation and occurrence-count method have classical antecedents; the present contribution is their explicit all-factor kernelization, local sign accounting, complete complexity reduction, and verified implementation. No general quasipolynomial bound for unrestricted diagrams is asserted.

Scope. Explicit, uncompressed Artin words; one-component closures; exact decisions with resource caps disabled. A small kernel is a theorem about a supplied presentation, not an assertion that every knot has an efficiently discoverable small presentation.

Contents

1	Problem, contribution, and relation to existing work	3
1.1	The bottleneck addressed here	3
1.2	What is classical, and what is developed here	3
1.3	A map of the results	3
2	Input model and elementary invariants	4
3	Singleton cuts and a closure-preserving genus kernel	5
3.1	The topological cut, with the necessary qualification	5
3.2	All cuts can be made at once	6
4	Local sign obstructions and the minority kernel	7
4.1	Using Bennequin locally rather than globally	7
4.2	What the kernel preserves, and what it does not	9
5	A complete exact algorithm and its bit complexity	9
5.1	From a kernel to a decision procedure	9
5.2	Why reduced characteristic-two homology suffices	10
5.3	A deliberately conservative complete bound	10
6	Amortized-linear singleton endpoint descent	12
6.1	Moves and the baseline cost	12
6.2	A deletion-only representation	12
6.3	Why the queue does not miss a free cancellation	13
6.4	A family with a genuine stage-complexity improvement	14
7	Sharpness, examples, and limits of the method	14
7.1	Presentation sensitivity	15
7.2	A precise remaining bridge to the general target	15
8	Implementation, certificate boundaries, and validation	16
8.1	Delivered modules and their contracts	16
8.2	Reference oracle and its limits	16
8.3	What is, and is not, independently verified	17
8.4	Finite verification corpus	17
9	Paired performance measurements	18
9.1	The endpoint stage	18
9.2	Negative controls	18
9.3	Controlled raw-cube comparison	19
10	Further research questions and integration priorities	19
11	Conclusion	21
A	Certificate formats and source binding	21
B	Reproduction and inspected source snapshot	22

1 Problem, contribution, and relation to existing work

1.1 The bottleneck addressed here

The inspected ProveIt implementation uses several inexpensive certificates before complete homology computation. Its braid-specific code tests the Bennequin inequality, has an exact at-most-three-strand branch, and can descend through singleton endpoint generators. The maintained recognition pipeline also includes geometric reduction, factorization, polynomial obstructions, and complete characteristic-two Khovanov backends. Its documented general guarantee remains exponential [9, 10]. A faster coefficient operation alone does not control the number of generators presented to such a backend. The present work therefore reduces the size of the topological input before constructing the chain complex.

The baseline endpoint helper is particularly concrete: it repeatedly cyclically free-reduces the whole word, scans both endpoint generator lists, rotates the word, and decreases the strand count. Its documented bound is $O(nb)$ word operations [11]. We replace this repeated traversal by a deletion-only data structure. Independently, we use *internal* singleton generators as connected-sum cuts. These cuts are not endpoint destabilizations and must not be justified as braid-group equalities.

1.2 What is classical, and what is developed here

Van Cott’s work on braid length already uses generator-occurrence arrays and the fact that a singleton occurrence splits a closed braid as a connected sum [1, proof of Theorem 5]. Thus neither that observation nor the underlying pigeonhole argument is claimed as a new topological discovery. Our construction uses every such cut without a primeness or shortest-word hypothesis, retains explicit factors, and gives local minority-sign accounting, bit-size bounds, verifiers, and an exact recognition reduction. Priority for the isolated $4k$ corollary has not been exhaustively established.

The imported inequality is the classical Bennequin inequality, used here only in its necessary condition for an unknot braid; Dasbach–Lin record this specialization [2]. The complete backend relies on Khovanov’s chain construction and the Kronheimer–Mrowka unknot-detection theorem [3, 4]. We do not claim that near-positive links have not previously been studied: Przytycki–Taniyama, for example, treat small numbers of negative crossings in much broader diagram classes [6].

Width restrictions and homological computation remain a different issue. Kelomäki–Schütz show, among other results, exponential behavior of a scanning approach even on certain three-braid families [7]. This cautions against inferring small chain complexes from small braid index. Our proof instead bounds the *entire explicit crossing input* to an exponential backend. The hierarchy route is also distinct: Lackenby’s July 2026 preprint explains in Section 9 that its described steps lack the precision needed for running-time estimates [8]. Nothing here supplies those missing hierarchy primitives.

1.3 A map of the results

There are three different conclusions, with different hypotheses.

First, singleton splitting is an unconditional closure-preserving reduction to $4g_B$ crossings. It uses no recognition invariant. Second, after local Bennequin tests, surviving instances have a $4k$ -crossing kernel. Rejected instances are already certified nontrivial. Third, replacing each residual factor by a complete exponential computation gives an unconditional parameterized recognition algorithm on explicit braid words. A quasipolynomial specialization follows only when the computed residual

parameter is polylogarithmic in the original input length.

The endpoint improvement has a separate status: its asymptotic time is linear on *all* valid inputs for that reduction routine, whether or not the routine finds anything. It is not a complete search for destabilizations, and linear running time of one stage is not linear running time of recognition.

2 Input model and elementary invariants

Definition 2.1 (Explicit Artin input). An input is a positive integer b and an explicit list

$$w = (a_1, \dots, a_n), \quad a_t \in \{\pm 1, \dots, \pm(b-1)\},$$

representing $\beta = \sigma_{|a_1|}^{\text{sgn}(a_1)} \dots \sigma_{|a_n|}^{\text{sgn}(a_n)} \in B_b$. The closure is required to have one component. The empty word in B_1 represents the unknot. Powers are expanded: $\sigma_i^{2^r}$ has 2^r letters in this model, not r bits.

Replace each signed letter by the adjacent transposition $(|a_t|, |a_t| + 1)$. The cycles of the resulting permutation correspond exactly to components of the closure. Thus the one-component promise is checked, not assumed by the public API. For a valid knot input every index $1, \dots, b-1$ occurs: an absent index separates two invariant strand sets. In particular $b \leq n+1$, a check made before allocating strand-indexed arrays.

Let

$$P = \#\{t : a_t > 0\}, \quad M = \#\{t : a_t < 0\}, \quad e = P - M, \quad k = \min(P, M) = \frac{n - |e|}{2}. \quad (1)$$

The canonical braid surface has b Seifert discs, n bands, and one boundary component. Consequently

$$g_B = \frac{n - b + 1}{2} \in \mathbb{N}. \quad (2)$$

This is also verified arithmetically by the permutation parity. It is the genus of the surface determined by this *word*, not necessarily the minimum genus of the knot. The same formula for a braid surface is standard [1]; our subsequent bounds never replace g_B by the smaller knot genus.

A machine word contains an index or counter of $O(\log(n + b + 2))$ bits. Statements about linear word operations use this model. A conservative bit simulation multiplies such bounds by a logarithmic factor. Parsing arbitrary external JSON is charged to its actual byte length; the normalized valid-word estimate does not pretend that a huge invalid integer can be read in constant time. No compressed-word algorithm is claimed.

Lemma 2.2 (Mirror invariance of the parameters). *Replacing every a_t by $-a_t$ reverses e , preserves n, b, g_B, k , and preserves whether the closure is the unknot.*

Proof. The letter counts P and M are exchanged. Reflection of the braid diagram gives the mirror knot, and the mirror of a spanning disc is a spanning disc. Reflection is involutive, so triviality is preserved in both directions. $\square$

3 Singleton cuts and a closure-preserving genus kernel

3.1 The topological cut, with the necessary qualification

Definition 3.1. An index $i \in \{1, \dots, b-1\}$ is a *singleton index* when

$$f_i := \#\{t : |a_t| = i\} = 1.$$

The count is unsigned: one occurrence of σ_i and one of σ_i^{-1} gives $f_i = 2$, not a singleton.

Lemma 3.2 (Singleton connected-sum cut). *Suppose $f_i = 1$. Delete that letter and project the other letters to indices $< i$ and $> i$, retaining their order. The projections define braids $\beta_L \in B_i$ and $\beta_R \in B_{b-i}$, the latter with indices shifted down by i . If $\widehat{\beta}$ is a knot, both projected closures are knots and*

$$\widehat{\beta} \cong \widehat{\beta}_L \# \widehat{\beta}_R.$$

Here $\cong$ denotes knot isotopy. This is not an equality in B_b .

Proof. Write the word as $u\sigma_i^\epsilon v$. Cyclically moving $u\sigma_i^\epsilon$ past the closure seam gives the conjugate word $vu\sigma_i^\epsilon$. No letter of vu has index i . Each remaining letter lies entirely on the first i strands or entirely on the last $b-i$ strands. The two alphabets commute, because any index from one differs from any index from the other by at least two. We may therefore arrange

$$vu\sigma_i^\epsilon = \alpha_L \alpha_R \sigma_i^\epsilon,$$

without changing the internal order of either projected word.

Close α_L and α_R separately in disjoint balls. In the displayed word the final σ_i^ϵ makes the standard local band connection between a short arc on the right edge of the left closure and a short arc on the left edge of the right closure. The band lies in the otherwise empty corridor between the two strand blocks. A sphere separating the balls can be adjusted across this band to meet the joined closure in precisely two points. The two capped pieces are the original projected closures. The local choice $\epsilon = \pm 1$ changes the drawn half-twist at the join, which can be absorbed by moving an attaching arc in its ball; it does not put a knotted tangle into the corridor. This is the usual geometric connected sum, not an arbitrary band sum.

For completeness, component counts also identify which case occurs. The permutations of α_L and α_R act on disjoint sets. Multiplication by $(i, i+1)$ merges the cycle containing i with the cycle containing $i+1$, and affects no other cycle. If their separate closure component counts are c_L, c_R , the joined count is $c_L + c_R - 1$. Since it is one, $c_L = c_R = 1$.

Finally, the projections of vu are cyclic rotations of the projections of uv . Projecting a rotated list simply moves its projected prefix to its end. Cyclic rotation preserves a closed braid, so the factors can be computed from the original linear word as stated. $\square$

This is the classical singleton observation in the explicit form needed by the verifier [1]. The restriction to the standard Artin corridor is essential to this proof. No claim about arbitrary Seifert-graph bridges, arbitrary bands between surfaces, or general band-generator encodings is being imported.

Lemma 3.3 (No cancellation of knot summands). *A finite connected sum of knots is the unknot if and only if every summand is the unknot.*

Proof. The connected sum of spanning discs gives a spanning disc, proving one direction. For the other, consider a decomposing sphere S meeting an unknot in two points, and an embedded spanning disc D transverse to S . The intersection consists of one arc joining the two boundary points and possibly circles. An innermost-circle surgery removes the circles without changing ∂D : choose a circle innermost on a puncture-free disc in S , replace the appropriate subdisc, and push off S . Such a puncture-free side exists because the intersection arc joins the two punctures without crossing that circle. Repetition leaves one intersection arc. Cutting D along it gives discs for the two capped knot arcs, so both summands are unknots. Induction proves the finite-sum assertion. $\square$

3.2 All cuts can be made at once

Let the singleton indices be $c_1 < \dots < c_s$, and set $c_0 = 0$, $c_{s+1} = b$. The strand intervals

$$I_j = \{c_{j-1} + 1, \dots, c_j\}, \quad 1 \leq j \leq s + 1,$$

partition all strands. Delete the s singleton letters. In each interval, retain every letter with $c_{j-1} < |a_t| < c_j$ and replace its index by $|a_t| - c_{j-1}$, preserving its sign. Let the resulting word have length n_j and $b_j = c_j - c_{j-1}$ strands.

Proposition 3.4 (Batch decomposition and exact accounting). *Every projected closure K_j is a knot, and*

$$\widehat{\beta} \cong K_1 \# \dots \# K_{s+1}, \quad (3)$$

$$n = s + \sum_j n_j, \quad b = \sum_j b_j, \quad (4)$$

$$g_B = \sum_j g_j, \quad g_j := \frac{n_j - b_j + 1}{2}. \quad (5)$$

Every generator index of a nonempty factor occurs at least twice. Empty factors are precisely one-strand unknot factors.

Proof. Apply Lemma 3.2 successively. Cutting at one singleton deletes no other occurrence and never changes the unsigned multiplicity of any remaining generator. Its left and right projections commute only across the deleted boundary, and later cuts are still singleton cuts within one factor. The cyclic rotations in the proof merely change the closure seams of projected words. Consequently the final factors are the displayed batch projections, up to those harmless rotations.

The first two counting identities follow from the partition and the deleted letters. There are $s + 1$ intervals, so

$$\sum_j (n_j - b_j + 1) = n - s - b + (s + 1) = n - b + 1,$$

which proves the genus identity. The original knot has no absent generator. Every non-cut generator therefore has multiplicity at least two, and its full list of occurrences stays within one factor. A factor on more than one strand cannot have an empty word because its permutation would not be a single cycle. $\square$

Theorem 3.5 (Linear-crossing genus kernel). *For every explicit knot braid, batch singleton splitting produces nonempty factors satisfying*

$$n_j \leq 4g_j, \quad b_j - 1 \leq 2g_j. \quad (6)$$

Their total crossing count m satisfies

$$m := \sum_{n_j > 0} n_j \leq 4g_B. \quad (7)$$

There is a single braid with the same knot closure, exactly m letters, and at most $2g_B + 1$ strands. All objects can be produced in $O(n + b)$ word operations, apart from the separately specified verifier.

Proof. For a nonempty factor all $b_j - 1$ generators occur at least twice; hence

$$n_j \geq 2(b_j - 1).$$

On the other hand, $n_j = 2g_j + b_j - 1$. Combining the two gives $b_j - 1 \leq 2g_j$, and substitution gives $n_j \leq 4g_j$. Summation and Proposition 3.4 prove (7).

To merge factors, use the standard overlapping-one-strand braid connected sum. Start with the first factor. Shift every index of the next factor by the previous strand count minus one and concatenate. Geometrically the two braids occupy consecutive height blocks and adjacent strand blocks with a single strand in common. Cutting that common strand between the blocks exhibits the ordinary connected sum. Iterating gives $1 + \sum_j (b_j - 1)$ strands and adds no letters. Empty one-strand factors change nothing. Thus the merged braid has at most $2g_B + 1$ strands and closure (3).

For production, count occurrences in an array indexed by the original generator, list the singleton indices in increasing order, and assign every non-cut generator to its interval. One pass over the original letters then appends them to their assigned factors with the appropriate index shift. Computing the interval assignments and all output lists costs $O(n + b)$ word operations in total. Merging needs only a running strand offset and another pass over the surviving letters. $\square$

The constructive argument avoids a possibly quadratic explicit sequence of commutations. The certificate proves the projected decomposition; it need not list every adjacent swap used in a conceptual isotopy proof. A single merged braid is useful for a standard kernelization interface, but keeping the factor list is better for recognition because the factors can be solved separately.

4 Local sign obstructions and the minority kernel

4.1 Using Bennequin locally rather than globally

The classical inequality implies that a b -strand braid representing the unknot has $e \leq b - 1$. Applying the same statement to the mirror gives

$$\widehat{\beta} = U \implies |e| \leq b - 1. \quad (8)$$

Only this specialization is used below [2]. It is one-sided: passing the inequality is not an unknot certificate.

For each nonempty factor produced by Proposition 3.4, let e_j be its exponent sum and $k_j = (n_j - |e_j|)/2$ its minority count. Define

$$\kappa = \sum_{n_j > 0} k_j, \quad k_* = \max_{n_j > 0} k_j, \quad g_* = \max_{n_j > 0} g_j, \quad (9)$$

where a maximum over no factors is zero.

Theorem 4.1 (Certified minority-sign kernel). *There is a polynomial-time procedure on explicit knot braids which has the following outcomes.*

- (i) *If a singleton factor satisfies $|e_j| > b_j - 1$, it returns a certificate that the input knot is nontrivial.*
- (ii) *Otherwise, it returns a single braid with the same knot closure, with m letters and b' strands, satisfying*

$$m \leq 4\kappa \leq 4k, \quad b' \leq 2\kappa + 1 \leq 2k + 1. \quad (10)$$

If there are no nonempty factors, it returns the empty one-strand unknot.

The compact braid payload has $O((k+1)\log(k+2))$ bits. For $k \geq 1$ its letter-list part has $O(k\log(k+2))$ bits. The retained audit certificate has size $O(n\log(n+b+2))$ bits and is not part of this compact-payload estimate.

Proof. A failing factor cannot be the unknot by (8). Lemma 3.3 then proves the original knot nontrivial.

If all factors pass, then

$$n_j - 2k_j = |e_j| \leq b_j - 1,$$

so $g_j = (n_j - b_j + 1)/2 \leq k_j$. Applying Theorem 3.5 factorwise gives $n_j \leq 4g_j \leq 4k_j$, and $b_j - 1 \leq 2g_j \leq 2k_j$. Summing proves the first inequalities in (10).

Let P_j, M_j be the signs retained in factor j . Since cuts only delete letters,

$$\sum_j \min(P_j, M_j) \leq \min\left(\sum_j P_j, \sum_j M_j\right) \leq \min(P, M) = k.$$

This proves $\kappa \leq k$ without assuming that different factors have the same majority sign. After merging, every generator index is at most $2k$ and there are at most $4k$ letters. Encoding the signed indices and strand count gives the asserted payload size. When $k = 0$, a passing instance has $g_j = 0$ for every factor, so it has no nonempty factor and the one-strand empty word suffices.

If a kernelization convention requires an ordinary braid instance rather than a rejection flag, output the fixed trefoil word $(1, 1, 1) \in B_2$ on rejection. Thus the decision-kernel size is at most $\max(3, 4k)$ letters. The closure-preserving claim applies to the surviving branch; replacing a rejected knot by the trefoil is only decision-equivalent. $\square$

Proposition 4.2 (Dominance and exact sign accounting). *If every singleton factor passes its Bennequin test, the original word passes its Bennequin test. Moreover, if e_S is the exponent sum of the s deleted singleton letters, then*

$$e = e_S + \sum_j e_j, \quad k - \kappa = \frac{s + \sum_j |e_j| - |e|}{2} \geq 0. \quad (11)$$

Proof. The exponent identity follows by partitioning the letters. Since $|e_S| \leq s$,

$$|e| \leq s + \sum_j |e_j| \leq s + \sum_j (b_j - 1) = b - 1.$$

Subtracting the definitions of k and κ , and using $n = s + \sum n_j$, gives the formula. Its nonnegativity also follows from the triangle inequality. Empty factors contribute zero throughout. $\square$

Consequently a separate global Bennequin check can be placed first as a cheap early rejection, but it does not find any rejection missed by all the local checks. The converse fails substantially.

Example 4.3 (Opposite signs do not cancel knot factors). Consider

$$\beta = \sigma_1^3 \sigma_3^{-3} \sigma_2 \in B_4.$$

Its exponent is 1, so $|e| \leq 3$ passes globally. Index 2 is a singleton. The two factors are the positive and negative two-strand trefoils, with exponents 3 and -3 and strand bound 1. Both fail locally, so the input is nontrivial. The two signs do not cancel connected-sum factors. The independent finite oracle in the package obtains reduced $\mathbb{F}_2$ -rank 9 for this seven-crossing example.

4.2 What the kernel preserves, and what it does not

The surviving factor list and merged braid preserve the *isotopy class of the closure*. Neither preserves the original braid element, strand count, writhe, or word length. The merged word may have factors with opposite majority signs, so its own minority count need not equal κ . What is true is that it retains exactly the non-cut signed letters and hence has minority count at most the original k . The length bound $m \leq 4\kappa$ also gives the independent estimate $k(\text{merged}) \leq 2\kappa$.

The full JSON returned by the implementation deliberately includes provenance and a replayable decomposition. That audit envelope can have size proportional to n even when the mathematical kernel is empty. The compact kernel object is the field `merged_kernel`, or a constant yes/no instance. Confusing these two encodings would incorrectly turn an $O(n)$ certificate into an asserted $O(k)$ output file.

The theorem is not a statement about a minimal number of negative crossings over all presentations. It uses the counts in the supplied word and the factors explicitly constructed from it. No conjugacy search, minimal braid-index computation, or optimization over Markov moves is hidden in the definition of the parameter.

5 A complete exact algorithm and its bit complexity

5.1 From a kernel to a decision procedure

Use the following procedure on a validated input. Form and verify the singleton decomposition; reject if any local Bennequin test fails; discard empty one-strand factors. For each remaining factor, run any complete exact unknot backend. Reject as soon as a factor is knotted, and accept only when every factor is proved trivial. Identical factor words may be memoized. If a configured resource cap stops a computation, record `UNKNOWN`; it is never treated as proof of triviality.

Correctness follows from Lemmas 3.2 and 3.3. It does not require the backend to output a homology basis, integral torsion, a connected-sum factorization, or a prime decomposition. Factorization is established *before* the backend is called.

5.2 Why reduced characteristic-two homology suffices

Lemma 5.1 (Transfer of unknot detection to total reduced $\mathbb{F}_2$ -rank). *For a knot K ,*

$$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) = 1 \iff K = U.$$

Proof. Kronheimer–Mrowka prove that total reduced rational rank one detects the unknot [4]. The integral reduced chain complex is finite and free as an abelian chain complex. The universal coefficient theorem, summed over all gradings, gives

$$\dim_{\mathbb{F}_2} \widetilde{\text{Kh}}(K; \mathbb{F}_2) = \dim_{\mathbb{Q}} \widetilde{\text{Kh}}(K; \mathbb{Q}) + 2t,$$

where t is the number of cyclic even-order torsion summands in integral reduced homology. Each such summand contributes once by tensor product and once in the adjacent homological degree by Tor; odd-order torsion contributes nothing. If the left side is one, the nonnegative integer t must be zero and the rational rank must be one. Conversely, the unknot’s reduced complex is a single free generator, so its reduced $\mathbb{F}_2$ -rank is one. Quantum shifts do not affect total rank. $\square$

This argument avoids relying on any unstated relationship between reduced and unreduced rank in a software backend. The supplied independent oracle computes the reduced complex directly, fixing the marked resolution circle to carry the label $x \in \mathbb{F}_2[x]/(x^2)$.

5.3 A deliberately conservative complete bound

Lemma 5.2 (Enhanced-state bound). *An m -crossing connected knot diagram has at most 4^m generators in its reduced enhanced-state cube. It can be recognized by this cube in time*

$$\text{poly}(m+1)2^{6m} \tag{12}$$

and space $\text{poly}(m+1)2^{4m}$ in a straightforward bit implementation.

Proof. Fix a complete resolution with c circles. Form the graph whose vertices are those circles and whose m edges are the original crossing sites, joining the circles incident to the site; loops are permitted. The original projection is connected, so this graph is connected. Thus $c - 1 \leq m$. The fixed marked circle leaves $2^{c-1} \leq 2^m$ labelings. There are 2^m resolution choices, giving at most $C = 4^m$ enhanced generators. The crossingless knot is the separate case $C = 1$.

Constructing resolutions and Frobenius edge maps costs a polynomial in m per local combinatorial operation. Each cube edge contributes at most two target labels for a source label. With dense binary columns, writing and updating the complete differential costs at most $\text{poly}(m+1)C^2$ bit operations. Ordinary Gaussian elimination over $\mathbb{F}_2$ uses at most $O(C^3)$ bit operations across the degree blocks: at most quadratically many column additions, each on at most C bits, suffices as a loose bound. Computing dimensions and differential ranks then gives homology dimensions. By Lemma 5.1, total reduced rank one is exactly the unknot case. Substituting $C \leq 4^m$ gives (12), and storing the matrices gives the space bound. $\square$

No attempt is made to optimize the exponent 6. The point is to establish a complete backend with a fully paid exponential cost, so that substituting a small kernel is legitimate. A fast scanner, sparse elimination, or the production at-most-three-braid certificate can improve actual execution without being needed for this upper bound. Bar–Natan’s fast-computation work provides the background for the more efficient scanning approach [5].

Theorem 5.3 (Complete parameterized recognition). *There is a deterministic exact algorithm for unknot recognition on explicit Artin braid knot inputs with*

$$T(n, \beta) \leq \text{poly}(n + 1) + \sum_{j: n_j > 0} \text{poly}(n_j + 1) 2^{6n_j}. \quad (13)$$

It has the following consequences:

$$T(n, \beta) \leq \text{poly}(n + 1) 2^{24g_*}, \quad (14)$$

$$T(n, \beta) \leq \text{poly}(n + 1) 2^{24k_*} \leq \text{poly}(n + 1) 2^{24k} \quad \text{if no local test rejects.} \quad (15)$$

For inputs rejected by a local test, preprocessing alone decides the instance. Thus the whole decision problem is fixed-parameter tractable in the supplied word's minority count k .

Proof. The kernel producer and independent verifier have polynomial bit complexity in $n + b$. Each factor is a validated knot braid and has an n_j -crossing closed diagram. Apply Lemma 5.2 separately to the factors. Correctness is the connected-sum criterion, so no multiplication of homology ranks is required for the decision.

Theorem 3.5 gives $n_j \leq 4g_j \leq 4g_*$. The number of factors and every polynomial prefactor in (13) are bounded polynomially in the original input length. This proves (14). When local tests pass, $g_j \leq k_j$ and $k_* \leq k$, proving (15). An early rejection costs only polynomial preprocessing, so it also fits a $\text{poly}(n + 1) 2^{O(k)}$ bound for every valid input. $\square$

Corollary 5.4 (Polynomial and quasipolynomial presentation classes). *With resource caps disabled, the algorithm is polynomial-time on families with $k_* = O(\log(n + 2))$ after passing local tests, and has running time*

$$n^{O(\log n)} = \exp(O(\log^2 n))$$

on families with $k_ = O(\log^2(n + 2))$. The same statements hold with g_* in place of k_* . In particular, the stronger but simpler input restriction $k = O(\log^2(n + 2))$ suffices.*

Proof. Substitute the stated parameter bounds into Theorem 5.3. The fixed polynomial preprocessing factor is absorbed in the corresponding polynomial or quasipolynomial expression. $\square$

Corollary 5.5 (Additive input and parameter costs). *The complete algorithm can be implemented with bit complexity*

$$O((n + 1) \log^2(n + 2)) + 2^{O(k)}.$$

In particular, for each fixed minority bound k , its input-length dependence is near-linear. More generally, a uniform bound on the residual maximum factor parameter gives near-linear input-length dependence even when there are many factors.

Proof. The array-based producer and occurrence-list/binary-search verifier cost $O((n + 1) \log^2(n + 2))$ bit operations, including the source and factor validation. No search over braid representatives occurs. On a passing input every nonempty factor has integer $g_j \geq 1$, hence $k_j \geq 1$. Thus the number of nonempty factors is at most $\kappa \leq k$, and their total length is at most $4k$. Their complete computations in (13) cost at most $k \text{poly}(k + 1) 2^{24k} = 2^{O(k)}$ for $k \geq 1$. The zero-parameter case and rejected inputs use preprocessing alone. For bounded k_* , each factor computation has a bounded cost and there are at most n factors. This last conclusion follows without computing or storing the product of their homology ranks. $\square$

These are complete, two-sided decisions on the stated classes, not merely invariant queries. They require no conjecture that a computed homological window captures all homology. They also do not become a general theorem by calling the parameter “small in practice”: a generic word can have $k = \Theta(n)$ and no useful cuts.

The maximum-factor bound can be much stronger than the total-parameter bound. A diagram with many independently certified bounded-size factors has bounded k_* even when κ grows linearly with its length. The decision remains a conjunction of small computations. It need not materialize an exponentially large product of homology groups or an exponentially large exact total rank.

6 Amortized-linear singleton endpoint descent

6.1 Moves and the baseline cost

Two closure-preserving moves are used. A cyclically adjacent pair $a, -a$ may be removed: if the pair straddles the current seam, first move the seam by conjugation. If the right endpoint index $b - 1$ appears once, write the word as $u\sigma_{b-1}^\epsilon v$, rotate to $vu\sigma_{b-1}^\epsilon$, and destabilize to $vu \in B_{b-1}$. The left endpoint index 1 has the symmetric move, followed by decreasing every surviving absolute generator index by one.

The existing helper stops once there are at most three strands and, otherwise, gives the right endpoint priority. The source-derived baseline reproduced in this package follows those choices. Its cost is $O(nb)$ because it can perform $b - 3$ rounds, each traversing and copying a word of length $O(n)$ [11]. It is correct as a sufficient-condition simplifier; no correction to its knot-theoretic semantics is needed.

6.2 A deletion-only representation

Label every input letter by its original position $0, \dots, n - 1$. Keep its signed generator immutable. Maintain:

- (a) a circular doubly linked list of live positions, a live-bit array, a head position, and a live length;
- (b) for each original generator index, its live occurrence count and the exclusive-or of the live position identifiers at that index;
- (c) a surviving strand interval $[\ell, h)$, initially $[0, b)$, and a queue of candidate inverse adjacencies.

The current left and right endpoint generator indices, in original coordinates, are $\ell + 1$ and $h - 1$. A surviving letter with original absolute index a has current index $a - \ell$. All relabeling is deferred until final output. A left destabilization increments ℓ ; a right destabilization decrements h .

A count equal to one makes the exclusive-or value the unique live position. Position zero is valid: the count, not a truth-value test on the exclusive-or, establishes uniqueness. Deleting a position updates its neighbors, count, and exclusive-or in constant word operations.

Initialize the candidate queue with all n positions, interpreted as left ends of cyclic adjacency pairs. On popping a live position, inspect its current successor. Remove the pair only if its signed letters are inverse. After deleting a pair, queue the predecessor of the removed pair, since that is the only newly created adjacency. Stale queue entries are harmless and discarded. Once the queue is empty, the word is cyclically freely reduced.

At more than the target number of strands, test the right endpoint count and then the left endpoint count. A singleton endpoint is removed, the head is set to its old successor to implement the specified cyclic rotation, and the new predecessor adjacency is queued. Repeat. If neither endpoint is a singleton, stop. When the strand target has been reached, drain the inverse queue before returning; no endpoint move beyond the target is required.

Theorem 6.1 (Linear endpoint descent with replay). *On a validated n -letter b -strand knot input, the preceding algorithm terminates in $O(n + b)$ word operations and $O(n + b)$ words of space. Its output has the same knot closure and has no greater word length or strand count. It produces at most n move records, occupying $O(n \log(n + b + 2))$ bits. A separate replay verifier checks the supplied moves in $O(n + b)$ word operations. A conservative bit bound for production and replay is*

$$O((n + b) \log(n + b + 2)).$$

Proof. Initially the circular list, counts, exclusive-or values, and endpoint interval represent the exact input. Removing a cyclic inverse pair represents cancellation in a conjugate braid word, so it preserves the closure and leaves the interval unchanged. The endpoint move has exactly the Markov interpretation described above. An endpoint generator is unique by its count, so no other surviving letter is removed or invalidated when the interval shrinks. Moving the head to the successor records the actual cyclic rotation, and deferred relabeling yields the correct final Artin indices. Induction proves the representation invariant and topological correctness.

Every successful cancellation deletes two positions, and every endpoint step deletes one. A position can be deleted only once. There are initially n queued candidates, and at most one new candidate is appended per successful deletion event. Hence at most $2n$ queue entries are popped. A pop costs constant word operations, including the test for a stale entry. There are at most n successful deletion events and at most one failed endpoint test after the final queue drain. More generally each successful drain followed by a further iteration contains an endpoint deletion, so the number of endpoint tests is also $O(n + 1)$. Array initialization and output cost $O(n + b)$. This proves the production bound.

A move record names original positions and, for destabilization, an endpoint side. Replay reconstructs its own circular list and counts. It verifies adjacency and opposite signs for cancellations; for destabilization it verifies the specified original position is live, has the required endpoint index, and has live count one. It then performs the local update and compares the claimed final word with the reconstructed word. Replay does not run the producer's search or trust its exclusive-or table. With at most n records, the replay time and space are linear as well. Each record and index has logarithmic bit length, giving the certificate and bit bounds. $\square$

The replay certificate proves the transformations it contains. It does not prove that all possible braid relations or destabilizations have been searched, and it need not even prove maximal descent under the chosen policy. The implementation's `queue_pops` field is a diagnostic; replay does not trust it as a proof of runtime.

6.3 Why the queue does not miss a free cancellation

Every initial adjacency is scheduled. If two live positions become neighbors later, at least one deletion event created that adjacency, and that event queued its predecessor. If a scheduled entry becomes stale or its successor changes before it is popped, either its relevant endpoint has disappeared or the later

change has scheduled the new adjacency. Thus after the queue empties no live inverse pair remains. This proves the structural invariant used between endpoint steps without rescanning the word.

Different orders of free cancellation can produce different linear seams, but cyclically reduced representatives of the same free-group conjugacy class are cyclic rotations of one another. Their generator counts agree. With the same right-before-left endpoint policy, this explains the expected agreement with the baseline up to cyclic rotation. Correctness of the new routine needs only its legal replay, not agreement with a particular baseline seam.

6.4 A family with a genuine stage-complexity improvement

Let

$$\alpha = \sigma_1 \sigma_2 \sigma_1 \sigma_2^{-1} \in B_3, \quad \beta_L = \alpha \sigma_3 \sigma_4 \cdots \sigma_{L+2} \in B_{L+3}. \quad (16)$$

The word α reduces by one braid relation and cancellation to $\sigma_2 \sigma_1$, whose closure is the unknot. Each additional positive letter is a Markov stabilization. For β_L ,

$$n = L + 4, \quad b = L + 3, \quad g_B = 1, \quad k = 1.$$

There are no free inverse cancellations in the displayed word. The baseline removes one right endpoint per round while traversing a word whose length decreases from $L + 4$ to 4. Its traversal work is

$$\sum_{r=1}^L \Theta(r + 4) = \Theta(L^2).$$

Theorem 6.1 gives $\Theta(L)$ time, including certificate replay, for the replacement. This is a strict asymptotic improvement of the endpoint stage on an infinite family, rather than a fitted timing exponent.

The family is *not* hard for every branch of the production recognizer. In particular, after descent the existing exact three-braid branch can decide it without homology. The stage improvement is meaningful precisely because it accelerates the route to that branch, not because the family requires a large Khovanov complex in an optimized recognizer.

7 Sharpness, examples, and limits of the method

Proposition 7.1 (Sharpness for the singleton-only reduction). *For every integer $r \geq 1$, there is an unknot braid on $2r + 1$ strands with $4r$ letters, $k = g_B = r$, and no singleton index. Thus both inequalities $m \leq 4k$ and $b' \leq 2k + 1$ are attained by the singleton-only rule on an infinite family of unknots.*

Proof. Take

$$\eta_r = \prod_{j=0}^{r-1} \left(\sigma_{2j+1} \sigma_{2j+2} \sigma_{2j+1} \sigma_{2j+2}^{-1} \right) \in B_{2r+1}. \quad (17)$$

Each block is the three-braid α from (16), shifted to its strand interval. Consecutive blocks overlap in one strand, so the closure is a connected sum of r unknots and hence an unknot. Every generator index occurs twice, every block contributes three positive and one negative letter, and

$$n = 4r, \quad e = 2r = b - 1, \quad k = r, \quad g_B = \frac{4r - (2r + 1) + 1}{2} = r.$$

The unique-occurrence rule makes no cut. The Bennequin bound is exactly saturated and does not reject. Thus the whole word survives as a $4r$ -crossing, $(2r + 1)$ -strand kernel. $\square$

This is an optimality statement for a specified preprocessing rule, not a lower bound on the problem or on all kernelizations. Each block admits a short Reidemeister-III/braid-relation reduction. The repository's rank-two and other certified rewrite stages are natural complements. It would be misleading to call (17) a hard family for the complete production pipeline.

Example 7.2 (Factorwise parameters versus total parameters). Place r copies of α on disjoint three-strand intervals, and connect consecutive intervals by one singleton generator each. For example, one can concatenate the internal words and then the $r - 1$ connecting letters. The closure is a connected sum of unknots, and the permutation argument in Lemma 3.2 verifies that it is a knot. The input has $3r$ strands and $5r - 1$ letters. With positive connecting letters, $k = g_B = r$. After cutting, all nonempty factors have $g_j = k_j = 1$, so $k_* = 1$ even though the total minority count is unbounded. Arbitrary shuffles of letters from disjoint blocks preserving each block's internal order give the same conclusion. This is a family on which the maximum-factor bound is substantially stronger than a bound written only in k .

7.1 Presentation sensitivity

One may insert any number of adjacent pairs $\sigma_i \sigma_i^{-1}$ into a braid word without changing its closure. Each insertion increases n by two, leaves e and b unchanged, and increases both k and g_B by one. This proves that these are not knot invariants. The linear free-reduction routine removes these particular examples immediately, so they do not demonstrate difficulty after all available preprocessing. They demonstrate only why a statement about a supplied presentation must not be silently replaced by a statement about the knot itself.

Similarly, the genus of an arbitrary Seifert surface of the knot is not the parameter in Theorem 3.5. A conversion from a general planar diagram or a band presentation to an Artin braid may change both the length and the minority count. A theorem extending the present complexity bound to such encodings must pay for that conversion and prove the relevant parameter bound. No such general conversion theorem is proved here.

7.2 A precise remaining bridge to the general target

To obtain a general $n^{O(\log n)}$ algorithm by this route, it would suffice to construct, in that time, a certified reduction of every input to factors with $g_* = O(\log^2 n)$, or to produce a conclusive certificate by another stage whenever such a reduction is unavailable. Our arguments do not establish that structural alternative. In particular, the existence of a tiny presentation of the unknot is not an algorithm for finding its isotopy from a complicated input.

There are two logically different directions for further progress: decrease the residual geometric parameter with costed, replayable moves, or make the exact backend substantially better than exponential in that parameter. The present results make the first direction measurable and prove one useful endpoint primitive linear; they do not resolve the general alternative.

8 Implementation, certificate boundaries, and validation

8.1 Delivered modules and their contracts

The package has no nonstandard runtime dependency. Its main components are:

File	Contract
<code>braidkernel/core.py</code>	Validated braid record, batch cut producer, independent decomposition verifier, merging, local Bennequin kernel.
<code>braidkernel/descent.py</code>	Linear cyclic free reduction and endpoint descent; independent original-position trace replay.
<code>braidkernel/cube.py</code>	Small-instance, complete exponential reduced $\mathbb{F}_2$ cube oracle; exact Laurent state sum for checks.
<code>braidkernel/recognizer.py</code>	Safe aggregation of kernel factors and exact backend decisions.
<code>integration/adapter.py</code>	Optional callback bridge to an existing safe backend, with a shared remaining time allowance.

The independent cut verifier uses occurrence lists and binary search for interval membership rather than the producer’s generator-owner table. It checks that every singleton is present in the certificate, recomputes all projections, validates their one-component closures, and checks the exact size and genus identities. Its bound is $O(n \log(b + 1) + b)$ word operations. Therefore the default *verified* kernel API is conservatively $O(n \log(n + 2))$ word time, not the producer’s linear bound. A bit simulation gives $O(n \log^2(n + 2))$ after normalized input validation.

The endpoint replay is different: every move is local, so producer plus independent replay retains the linear word bound. This distinction is preserved in the measurements.

The statuses are semantically separated. **CORE** means a surviving instance requiring an exact decision; it is not acceptance. **UNKNOT** and **KNOTTED** mean established decisions within the imported topology and the verified computation. **UNKNOWN** means that the requested resource-bounded computation did not settle the instance. Agreement with a Jones value is never turned into acceptance.

8.2 Reference oracle and its limits

The oracle resolves every crossing and labels circles by $1, x \in \mathbb{F}_2[x]/(x^2)$. The marked circle, chosen by an input strand point, is always labeled x . Its local maps are

$$\begin{aligned} \mu(1 \otimes 1) &= 1, & \mu(1 \otimes x) &= x, & \mu(x \otimes 1) &= x, & \mu(x \otimes x) &= 0, \\ \Delta(1) &= 1 \otimes x + x \otimes 1, & \Delta(x) &= x \otimes x. \end{aligned}$$

In characteristic two cube signs disappear. A column is stored as an arbitrary-precision integer bit vector. The oracle reports raw homological degrees, chain dimensions, differential ranks, total reduced rank, and optional explicit checks that $d^2 = 0$. It does not report quantum grading or integral torsion, neither of which is needed for the total-rank decision theorem.

The default caps are 12 crossings and 200,000 enhanced generators *per factor*. The theoretical complete algorithm is obtained by setting both caps to **None** and imposing no time cap. The reference implementation is intended for audits and small kernels; it is not a replacement for an optimized production homology engine on large cores.

Time limits are cooperative. The oracle checks elapsed time at construction stages and before matrix reductions, not at every individual bit-vector operation. The adapter checks the remaining global allowance before calls, and preprocessing does not promise hard real-time interruption. Exhaustion cannot produce a false unknot decision, but a small requested wall-time allowance is not a guaranteed hard upper bound. Production integration should add the host’s cooperative budget callback inside long loops.

8.3 What is, and is not, independently verified

Certificate replay independently validates the finite combinatorial transformation. It is not a machine-checked proof of the topological singleton lemma, Markov’s theorem, the Bennequin inequality, or Khovanov unknot detection. Those mathematical inputs remain in the trusted theorem base. The human-readable proofs establish the newly assembled reduction and cost accounting; no Lean or Coq formalization is supplied.

The reduced cube is independent of the repository’s scanner. The validation also compares short three-braids with a separate matrix criterion documented in the inspected upstream source [10]. This is useful differential testing, not a new proof of the three-braid classification. The Laurent bracket checker uses independent state-sum algebra, but shares the resolution-connectivity routine with the cube; it is therefore not fully independent diagram geometry.

Only specified upstream source files were inspected at commit `8388b5368036`; a complete checkout was not executed in this environment. No full upstream regression suite or production recognition benchmark was run, and no remote repository was modified. The callback adapter was tested with controlled stand-ins, not an installed `fastunknot`. Integration requires its own full-suite and corpus validation.

8.4 Finite verification corpus

The final recorded unit run passed **42 test methods**. Beyond individual examples, the tests cover corrupted certificates, malformed words and link rejection, mirrors, empty inputs, generator caps, safe unknown propagation, large stabilization chains, and callback contracts. The test log is `data/tests.txt`.

The independent corpus run is summarized in Table 1. Exhaustive enumeration considers every three-braid word of lengths 2, 4, and 6 over $\{\pm 1, \pm 2\}$, retaining only one-component closures. For each retained word, the original reduced cube is computed, d^2 is checked, the matrix criterion is compared, the kernel verdict is checked, and the reduced-rank product of the certified factors is compared with the original rank.

The planted internal-cut corpus uses two four-letter three-braid knot words, shifts one onto the other three strands, shuffles the two internal words while preserving each internal order, and inserts a single positive or negative connecting generator. These are nine-crossing six-braid knot inputs. Original, projected, and merged reduced ranks agree; original and merged normalized Laurent brackets agree. The expected rank product also follows from the reduced connected-sum chain construction and the field Künneth theorem. The decision algorithm itself does not depend on this product check.

The random endpoint comparison corpus contains valid knot braids with 2–10 strands and up to 60 letters. For each, the new trace is replayed, and the terminal strand count, length, and cyclic word are compared with the source-derived baseline. Exact seeds, dimensions, and short-word records are in

Table 1: Completed finite validation. These counts are tests, not formal verification or an exhaustive classification of arbitrary braids.

Check population	Count
Candidate three-braid words	4368
Retained three-braid knot closures	2856
Planted internal-cut knot inputs	32
Original cubes explicitly checked for $d^2 = 0$	2888
Random new/old endpoint comparisons	1000
Observed mismatches	0

`data/validation_summary.json` and `data/validation_cases.json`. These finite checks support implementation confidence; they do not replace the asymptotic or topological proofs.

9 Paired performance measurements

9.1 The endpoint stage

The endpoint experiment compares the source-derived old helper with the new producer *plus its independent verifier*. It uses the stabilization family (16), five randomized paired rounds, and a second identical baseline arm as an A/A control. Each call builds fresh work arrays. The timing excludes imports, parsing, and command-line startup. The terminal result is checked for every call.

Table 2 reports the saved run on CPython 3.13.5. The ratio is the median of paired baseline/new ratios, not the ratio of the two median times. The exact platform description and all samples are retained in `data/benchmark.json`. These are host-dependent descriptive measurements, not hardware-independent guarantees.

Table 2: Endpoint stage: old helper versus new production plus replay. Times are milliseconds. Larger old/new ratios indicate a speedup.

Strands	Letters	Old median	New median	Old/new	A/A
64	65	0.447	0.126	3.61	1.003
256	257	6.359	0.394	15.60	1.029
1024	1025	107.844	1.833	60.57	1.003
4096	4097	1901.152	7.481	238.45	1.002

The largest case has 4096 strands and 4097 letters. Its measured paired ratio is approximately 238.45. The reason to expect improvement with input size is Theorem 6.1 and the quadratic baseline analysis, not extrapolation of that single number. At the smallest measured stabilization size the ratio is only about 3.61, illustrating the influence of fixed costs.

9.2 Negative controls

For a tiny no-cut three-braid core and an internal-cut-only four-braid input, 15 paired rounds were run in batches of 50 calls to resolve short timings. The old/new ratios were approximately 0.152 and

0.204. Thus the new *verified* endpoint routine was about 6.6 and 4.9 times slower, respectively, on these tiny controls. The baseline quickly returns, whereas the new path builds and replays additional data structures. An internal cut does not itself help an endpoint-only routine.

These observations argue against an unconditional default replacement without production measurements. A sensible integration can bypass descent once $b \leq 3$, since the exact small-braid decision branch already applies, and can make the full verified stage opt-in while measuring larger inputs. Such a policy change needs separate measurement; the table does not retrospectively credit an optimization that was not run.

9.3 Controlled raw-cube comparison

A separate experiment contrasts the package’s own raw cube with its own kernel followed by the *same* cube. It uses three paired rounds on stabilized unknots. Table 3 records the original raw enhanced-generator counts and paired ratios.

Table 3: Standalone reference experiment, not the production recognizer. The baseline deliberately skips all existing geometric and braid shortcuts.

Original letters	Raw enhanced generators	Raw/kernel time ratio
6	297	8.45
8	2673	105.60
10	24057	1146.56

This measures the effect of avoiding unnecessary cube expansion. It must not be advertised as a corresponding speedup over ProveIt’s complete pipeline: that pipeline already has a three-braid shortcut and does not need to compute this raw cube. The stage-level endpoint comparison is the direct evidence for replacing the old endpoint primitive. All other production-speed claims remain to be measured with the entire recognizer enabled and with identical resource policies.

10 Further research questions and integration priorities

Research question 10.1 (Certified low-parameter alternatives). Can every diagram be reduced, within $n^{O(\log n)}$ time, either to a conclusive certificate from another stage or to explicit braid factors with $g_* = O(\log^2 n)$?

This is a sufficient bridge to the general target, not a theorem proved by the current kernel. It needs bounds both on the search and on intermediate certificate size. An existence argument for a short unknot representative does not pay either cost. An initial tractable goal is a theorem for a well-defined family of braid presentations, with an independently checkable reduction history.

Research question 10.2 (Composition with rank-two and braid-relation reductions). Can local certified braid-relation moves be scheduled so that each accepted move either lowers g_B , exposes a singleton cut, or decreases a bounded auxiliary potential?

The sharp family (17) is an explicit benchmark: one relation per block defeats the singleton barrier. The challenge is a global amortization that charges failed searches and avoids repeatedly rescanning

unchanged regions. This would combine naturally with the repository’s existing rank-two and causal move-search work, without assuming that every search branch succeeds.

Research question 10.3 (Improving the crossing constant). What crossing bound below $4k$ is achievable by a polynomial-time, closure-preserving kernel with a specified richer move set?

Proposition 7.1 rules out improvement only for the present singleton-only output rule. It places no obstruction on a kernel that contracts rank-two patterns, uses small-braid decisions to remove proved unknot factors, or recognizes other certified local replacements. Any claimed lower bound must specify the allowed reductions and distinguish crossing count from encoded bits.

Research question 10.4 (Small-multiplicity interfaces). Can a generator with a few occurrences be replaced by a bounded-complexity tangle interface rather than a connected-sum cut?

A singleton creates a two-point separating sphere after closure. Two or more occurrences generally do not. A correct extension must retain boundary connectivity and the state needed for future gluing; merely multiplying factor answers is unsound. The useful target is a compositional representation whose size depends singly exponentially on interface complexity, with no hidden generator multiplicity explosion.

Research question 10.5 (Input conversion with a paid sign parameter). Which classes of planar diagrams admit efficiently constructible Artin braid representatives whose residual k_* or g_* is controlled by a comparably small diagram parameter?

The algorithmic input model matters. One should separately count conversion time, output letters, and introduced positive/negative pairs. Generalized band generators and binary-encoded twist powers require their own theorems; expanding them first may destroy the intended bound.

Research question 10.6 (Parameter-aware backend selection). Can the exact residual computation be selected using (n_j, b_j, k_j, g_j) and certified scan parameters, with a provable cost no worse than the best of several complete backends up to controlled overhead?

A practical design could race a small-braid certificate, the maintained sparse scanner, and a suitably capped cube or external exact backend. Every racer must preserve the same decision semantics and pass unused budget consistently. A large incomplete rank or a matching polynomial is not a decision. An asymptotic theorem must include the total cost of all racers, not just the winning one.

Research question 10.7 (Production ablation and certificate compatibility). Where in the actual recognizer should local cuts and linear endpoint descent run, and which existing shortcuts should precede them?

The first implementation milestone is an opt-in stage with a versioned trace schema and source-bound replay. It should preserve old certificate verification, validate the source braid against its diagram, and retry all affected factor computations under one global budget. Paired full-pipeline measurements must include no-cut cores, inputs already settled by earlier filters, internal connected sums, long stabilization chains, and resource-exhaustion cases. The delivered measurements do not yet answer this integration question.

Research question 10.8 (Formal verification and streaming certificates). Can the finite permutation, projection, and size accounting be formalized, followed by a proof-carrying interface to the topological singleton lemma?

The occurrence-array and linked-list invariants are natural first targets. A useful formal artifact would establish that every accepted trace denotes a legal sequence, rather than merely reproducing the producer. For very large words, one can also study streaming output of moves and factors, while preserving enough source information for independent replay. The compact kernel and the linear-size audit record should remain distinct objects.

11 Conclusion

The resulting package gives two concrete advances for the repository’s algorithmic program. It exposes a complete, computable minority-sign parameter through a certified $4k$ -crossing reduction, and it replaces a quadratic-on-stabilizations endpoint routine by a linear one with replay. The parameterized theorem gives genuine two-sided quasipolynomial recognition when the residual maximum factor parameter is $O(\log^2 n)$; it does not assume an unproved statement about homological windows or template multiplicities.

The scope is equally concrete. The singleton observation is classical, the kernel is presentation-dependent, the independent oracle is exponential, and the full production pipeline has not been benchmarked or integrated in this work. The main remaining research task is a costed structural reduction that forces small residual factors more generally, or a stronger exact method for the factors that remain.

A Certificate formats and source binding

The `artin-singleton-cuts-v1` certificate records the original strand count and word length, then all cut generator indices, their original positions, and their signed letters. Each factor record contains a half-open zero-based strand interval `[start,stop)`, its strand count, and its projected signed word. Replay receives the *actual original braid* as an argument. The dimensions alone are not treated as authentication; all cuts and projected words are recomputed from that braid. Obstruction indices in the kernel result refer to the separately listed *nonempty* factors, while the decomposition certificate retains empty intervals too.

The `linear-endpoint-descent-v1` certificate records a sequence of two move types:

```
{"op": "cancel", "first": ORIGINAL_ID, "second": ORIGINAL_ID}
{"op": "destabilize", "side": "left" | "right",
 "position": ORIGINAL_ID}
```

It also records the final cyclic ordering of original position identifiers and the final relabeled braid. The verifier rejects deleted, out-of-range, or nonintegral identifiers, nonadjacent cancellations, wrong endpoint sides, non-singleton deletions, and inconsistent final words. It verifies equivalence, not maximality of the search. Diagnostics such as queue counts are not trusted proof fields.

Neither certificate should be attached to a different intermediate diagram without replaying the preceding transformations. A production integration that starts from a planar diagram must establish that its cached source braid actually represents that diagram before applying the braid certificate. The callback adapter assumes this binding is supplied by the caller; it does not infer it from matching crossing counts.

B Reproduction and inspected source snapshot

Run the following from the package root:

```
python -m unittest discover -s tests -v
python scripts/validate.py
python scripts/benchmark.py
python scripts/render_tables.py
sh article/build.sh
python -m braidkernel examples/local_bennequin.json
python -m braidkernel examples/stabilized_unknot.json --kernel-only
```

The complete, uncapped reference decision can be invoked from Python:

```
from braidkernel import Braid, recognize
answer = recognize(Braid.checked(b, word),
                  max_crossings=None, max_generators=None)
```

That setting removes safety caps and can require exponential resources. The normal examples and unit tests use small kernels and bounded computations.

The source audit is pinned to ProveIt commit

8388b53680366eed81f8bc23d8db8cfbf2cd956e.

The relevant files are under `Topology/UnknotRecognition/fast/fastunknot/`. Their inspected blob identifiers are:

```
braid_reduction.py
    974d42c8c722018fad7678b2000758155ffe5162.
```

```
braid.py
    d7de8c2d8f7b12f35a4ccc19478ca09a370bc2a3.
```

```
recognize.py
    6a2db8e7c7a667a4d830ff0527992fcf4c231962;
```

the initial pipeline contract and definitions were inspected, not every implementation branch.

The bundled endpoint baseline is a source-derived reproduction of the relevant helper, not a claim that a full upstream checkout was executed. Third-party paper PDFs are cited, not redistributed. The artifact manifest records hashes of the delivered source, article, tests, examples, and saved measurements.

References

- [1] C. A. Van Cott, *Relationships between braid length and the number of braid strands*, Algebraic & Geometric Topology **7** (2007), 181–196. <https://doi.org/10.2140/agt.2007.7.181>.
- [2] O. T. Dasbach and X.-S. Lin, *The Bennequin number of n -trivial closed n -braids is negative*, arXiv:math/0010278 (2000). <https://arxiv.org/abs/math/0010278>.
- [3] M. Khovanov, *A categorification of the Jones polynomial*, Duke Mathematical Journal **101** (2000), 359–426; arXiv:math/9908171. <https://arxiv.org/abs/math/9908171>.
- [4] P. B. Kronheimer and T. S. Mrowka, *Khovanov homology is an unknot-detector*, arXiv:1005.4346 (2010). <https://arxiv.org/abs/1005.4346>.
- [5] D. Bar-Natan, *Fast Khovanov homology computations*, arXiv:math/0606318 (2006). <https://arxiv.org/abs/math/0606318>.
- [6] J. H. Przytycki and K. Taniyama, *Almost positive links have negative signature*, arXiv:0904.4130 (2009). <https://arxiv.org/abs/0904.4130>.
- [7] T. Kelomäki and D. Schütz, *On computational complexity of Khovanov homology*, arXiv:2601.02119 (2026). <https://arxiv.org/abs/2601.02119>.
- [8] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1 (25 July 2026), especially Section 9. <https://arxiv.org/abs/2607.23350>.
- [9] V. Reshetnikov, *ProveIt: Topology/UnknotRecognition*, repository and maintained research documentation, inspected 8 October 2026. <https://github.com/VladimirReshetnikov/ProveIt/tree/8388b53680366eed81f8bc23d8db8cfbf2cd956e/Topology/UnknotRecognition>.
- [10] ProveIt contributors, *fastunknot/braid.py*, source at commit 8388b5368036. <https://github.com/VladimirReshetnikov/ProveIt/blob/8388b53680366eed81f8bc23d8db8cfbf2cd956e/Topology/UnknotRecognition/fast/fastunknot/braid.py>.
- [11] ProveIt contributors, *fastunknot/braid_reduction.py*, source at commit 8388b5368036. https://github.com/VladimirReshetnikov/ProveIt/blob/8388b53680366eed81f8bc23d8db8cfbf2cd956e/Topology/UnknotRecognition/fast/fastunknot/braid_reduction.py.